

1. Entornos virtuales con venv

Un entorno virtual es una carpeta que contiene un intérprete de Python aislado y su propio directorio de paquetes (site-packages). Permite que cada proyecto tenga sus dependencias y versiones sin interferir con otros proyectos ni con el Python del sistema.

El módulo venv viene incluido en la biblioteca estándar. Para crear un entorno en la carpeta .venv del proyecto:

```
python -m venv .venv
```

Para activarlo:

Windows (PowerShell): `.venv\Scripts\Activate.ps1`

Windows (cmd): `.venv\Scripts\activate.bat`

Linux y macOS: `source .venv/bin/activate`

Al activarlo, el prompt muestra el nombre del entorno y los comandos python y pip pasan a apuntar al intérprete del entorno. Para salir se ejecuta deactivate. Activar no es obligatorio: también se puede invocar directamente `.venv/bin/python` (o `.venv\Scripts\python.exe` en Windows).

Si PowerShell bloquea el script de activación por la política de ejecución, se puede habilitar para el usuario actual con: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

La carpeta del entorno virtual nunca se sube al repositorio: se agrega `.venv/` al archivo `.gitignore`, porque contiene binarios específicos del sistema operativo y se puede reconstruir en cualquier momento a partir de la lista de dependencias.

2. Instalar dependencias con pip y requirements.txt

pip es el instalador de paquetes de Python y descarga paquetes desde PyPI. Siempre conviene ejecutarlo como módulo del intérprete del entorno, así se evita instalar en otro Python por error:

```
python -m pip install requests
python -m pip install "requests>=2.31,<3"
python -m pip install --upgrade requests
python -m pip uninstall requests
```

El archivo requirements.txt lista las dependencias del proyecto, una por línea, con especificadores de versión opcionales (==, >=, <, ~=). Para instalar todas:

```
python -m pip install -r requirements.txt
```

pip freeze muestra todos los paquetes instalados con su versión exacta, incluidas las dependencias transitivas. Con python -m pip freeze > requirements.txt se genera un archivo con versiones fijadas (pinning), lo que hace reproducible la instalación en otra máquina o en un servidor.

Buenas prácticas: fijar versiones exactas en aplicaciones que se despliegan, y usar rangos compatibles en bibliotecas que otros van a instalar. python -m pip list --outdated muestra los paquetes con versiones nuevas y python -m pip check verifica que no haya dependencias incompatibles entre sí.

3. pyproject.toml y herramientas modernas

El archivo pyproject.toml (PEP 518 y PEP 621) es el estándar actual para describir un proyecto de Python: nombre, versión, versión mínima de Python y dependencias, además de la configuración de herramientas como pytest, Ruff o mypy en secciones [tool.*].

```
[project]
name = "mi-proyecto"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = ["requests>=2.31", "pydantic>=2"]
```

Con ese archivo, `python -m pip install -e .` instala el proyecto en modo editable junto con sus dependencias.

Existen herramientas que automatizan la gestión de entornos y dependencias. uv (escrita en Rust) crea entornos con `uv venv`, instala con `uv pip install` y administra proyectos completos con `uv add` y `uv sync`, generando un archivo de bloqueo (`uv.lock`) con las versiones exactas de todo el árbol de dependencias. Poetry y PDM ofrecen flujos similares con su propio archivo de bloqueo.

Recomendación general: un entorno virtual por proyecto, dependencias declaradas en un archivo versionado (`requirements.txt` o `pyproject.toml`), versiones fijadas para lo que se despliega y el entorno siempre fuera del control de versiones.